DATA305 / DATA475 · Machine Learning Methods

Data Pipelines & Preprocessing

tf.data API · Normalization · Augmentation · Keras Preprocessing Layers

Lecture 6

Week 7 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda

Five sections · CIFAR-10 experiments · Oxford Flowers pipeline examples

01 tf.data Pipeline

map · shuffle · batch · prefetch · cache

02 Normalization

÷255 · z-score · per-channel · Keras layer

03 Data Augmentation

Geometric · colour · training vs inference

04 Keras Preprocessing

Where to put it · Oxford Flowers · TFRecord

05 CIFAR-10 Experiments

Pipeline throughput · normalization effect · augmentation regularization on CNN

Experiments: keras.datasets.cifar10 · Pipeline examples: oxford_flowers102 · Geron Ch. 13 pp. 503–541

Lecture 5 → Lecture 6

In L4–L5 we focused on the model side: gradient stability, optimizers, schedules, and regularization.
In L6 we move to the data side: efficient input pipelines, preprocessing strategies, and augmentation.

Lectures 4 & 5 — Model Side

- Vanishing & exploding gradients
- He & Glorot initialisation
- Batch Normalization
- Adam, AdamW optimizers
- LR Scheduling: warmup, cosine
- Regularization: Dropout, L1/L2

Lecture 6 — Data Side

- Build efficient tf.data pipelines
- Normalize inputs for stable training
- Apply augmentation as data-side regularization
- Use Keras preprocessing layers in-model
- Profile and optimize throughput (samples/sec)
- Connect data pipeline to full training loop

Good training dynamics (L5) + efficient data pipelines (L6) = the complete recipe for reliable model training.

01

The tf.data Pipeline

Dataset sources · map · shuffle · batch · prefetch · cache

Why tf.data? — The GPU Starvation Problem

Without a pipeline, the GPU sits idle while the CPU loads and preprocesses the next batch.

Naive (no pipeline):

← GPU idle during load + preprocess

prefetch() overlaps CPU prep of batch N+1 with GPU training of batch N → no idle time

With prefetch (pipelined):

↑ *no idle time — CPU prepares next batch while GPU trains current one*

- Lazy evaluation — data loaded only when needed, not all at once in RAM
- Parallel map() — multi-threaded preprocessing across all CPU cores
- Works with tensors, NumPy arrays, files on disk, TFRecord, generators
- Integrates directly with model.fit() — no custom training loop needed

Creating a tf.data.Dataset


```python
# From NumPy arrays (CIFAR-10 style)
ds = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)
)

ds = (
    ds.map(
        preprocess_fn,
        num_parallel_calls=tf.data.AUTOTUNE
    )
    .shuffle(buffer_size=1000)
    .batch(batch_size=32)
    .prefetch(tf.data.AUTOTUNE)
)
```

```python
# From tensorflow_datasets
import tensorflow_datasets as tfds

ds, info = tfds.load(
    'oxford_flowers102',
    split='train',
    as_supervised=True,
    with_info=True,
)

# info.splits['train'].num_examples → 6552
# info.features['label'].num_classes → 102
```

Pipeline Operations — map · shuffle · batch · prefetch

`map(fn, num_parallel_calls=AUTOTUNE)`

When: Any per-sample transform: decode, resize, cast, augment.

```
map(lambda x,y: (resize(x, [224,224]), y))
```

`shuffle(buffer_size=N)`

When: Randomise training order each epoch. Rule: N ≥ batch_size × 10.

```
shuffle(1000)   # not needed at inference
```

`batch(batch_size, drop_remainder=False)`

When: Always. Should be the last step before prefetch.

```
batch(32)   # or 64 / 128 / 256
```

`prefetch(buffer_size=AUTOTUNE)`

When: Always last. Overlaps CPU work with GPU training.

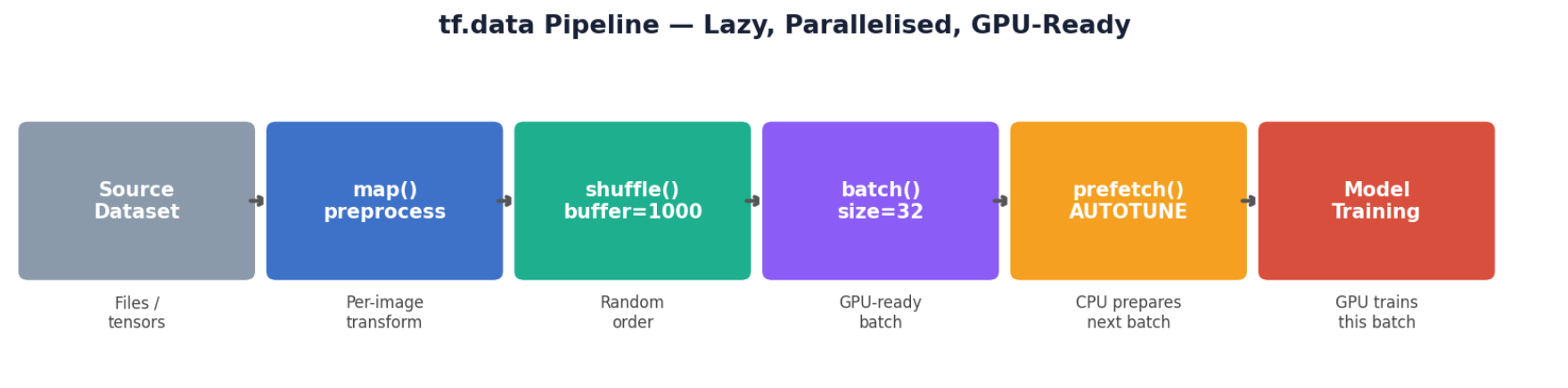
```
prefetch(tf.data.AUTOTUNE)  # ← always add
```

`cache()` — AFTER decode/resize, BEFORE augmentation

When: Caches deterministic data; augmentation runs fresh each epoch on cached images.

```
...
.cache().shuffle(N).map(augment).batch(B).
prefetch(...)
```

Putting It Together — Full CIFAR-10 Pipeline




```python
# Complete CIFAR-10 training pipeline
AUTOTUNE = tf.data.AUTOTUNE

train_ds = (
  tf.data.Dataset
    .from_tensor_slices((x_tr, y_tr))
    .map(preprocess,
         num_parallel_calls=AUTOTUNE)
    .cache()   # after deterministic steps
    .shuffle(buffer_size=1000)
    .batch(32)
    .prefetch(AUTOTUNE)  # always last
)
```

```python
# Validation pipeline
# (no shuffle, no augmentation)
val_ds = (
  tf.data.Dataset
    .from_tensor_slices(
        (x_val, y_val))
    .map(preprocess,
         num_parallel_calls=AUTOTUNE)
    .batch(32)
    .prefetch(AUTOTUNE)
)
```

02

Normalization

z-score · min-max · per-channel · Keras Normalization layer

Why Normalize? — Convergence Speed and Stability

Normalization transforms pixel values so gradient magnitudes are consistent across features — enabling stable, fast convergence.

✗ **Raw pixels (0–255): 10.4% val**

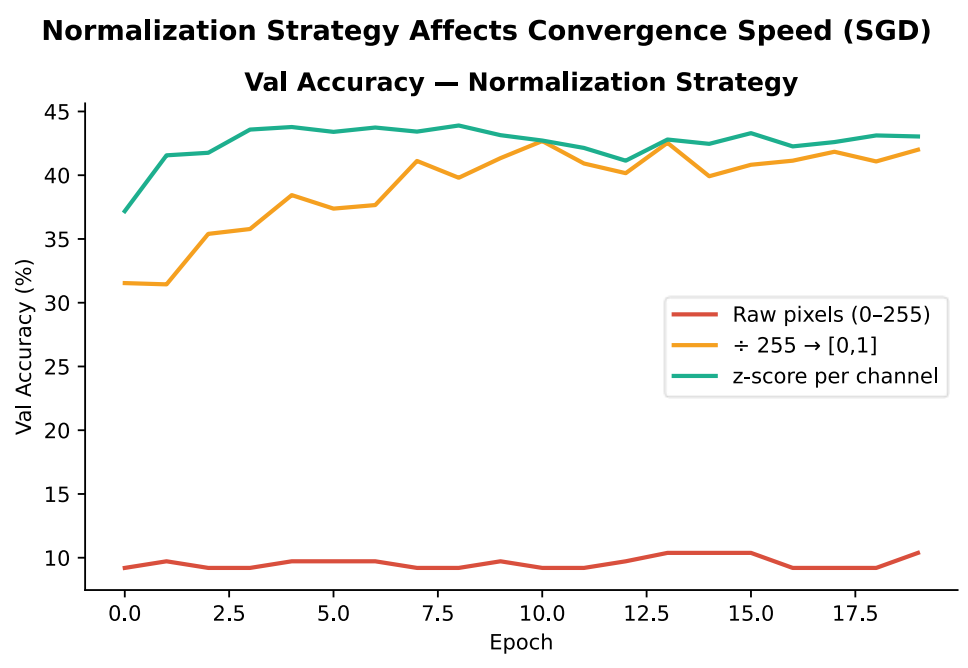
Gradient magnitudes too large for SGD (lr=0.05). Optimizer oscillates — model barely learns.

~ **÷ 255 → [0, 1]: 42.7% val**

Convergence is stable and much faster. However, data is not centered (mean ≠ 0).

✓ **z-score per channel: 43.9% val**

Mean≈0, std≈1 per channel. Fastest convergence. The ImageNet standard for pretrained backbones.



CIFAR-10, SGD lr=0.05, MLP without BN, 12k training images

Normalization Strategies

÷ 255 (pixel rescale)

$$x' = x / 255.0 \rightarrow [0, 1]$$

Pros:

✓ Simple, fast, no statistics needed

✓ Works well for images with consistent lighting

Cons:

✗ Does not center data (mean ≠ 0)

✗ Slower convergence than z-score

```
x = x.astype('float32') / 255.0
```

z-score (standardization)

$$x' = (x - \mu) / \sigma \rightarrow \text{mean} \approx 0, \text{std} \approx 1$$

Pros:

✓ Centers data — mean=0 helps optimizers

✓ Per-channel μ/σ is standard for ImageNet

Cons:

✗ Requires computing stats on training set

✗ Sensitive to outliers in σ

```
Normalization(axis=-1)  # per-channel
```

min-max (feature scaling)

$$x' = (x - x_min) / (x_max - x_min) \rightarrow [0, 1]$$

Pros:

✓ Bounded range — convenient for some activations

✓ No centering needed for certain tasks

Cons:

✗ Sensitive to outliers (min/max can shift)

✗ Not centered — may slow convergence

```
Rescaling(scale=1/127.5, offset=-1)  # →[-1,1]
```

Keras Normalization Layer — Per-Channel Statistics


```python
# Step 1: create and adapt the layer
norm_layer = tf.keras.layers.Normalization(
    axis=-1  # per last axis = per channel
)
norm_layer.adapt(
    x_train.astype('float32')
)
# Layer now stores μ and σ per channel

# Step 2a: use in tf.data pipeline
train_ds = (
    tf.data.Dataset
    .from_tensor_slices((x_tr, y_tr))
    .map(lambda x, y: (
        norm_layer(tf.cast(x, tf.float32)), y
    ))
    .batch(32)
    .prefetch(tf.data.AUTOTUNE)
)
```

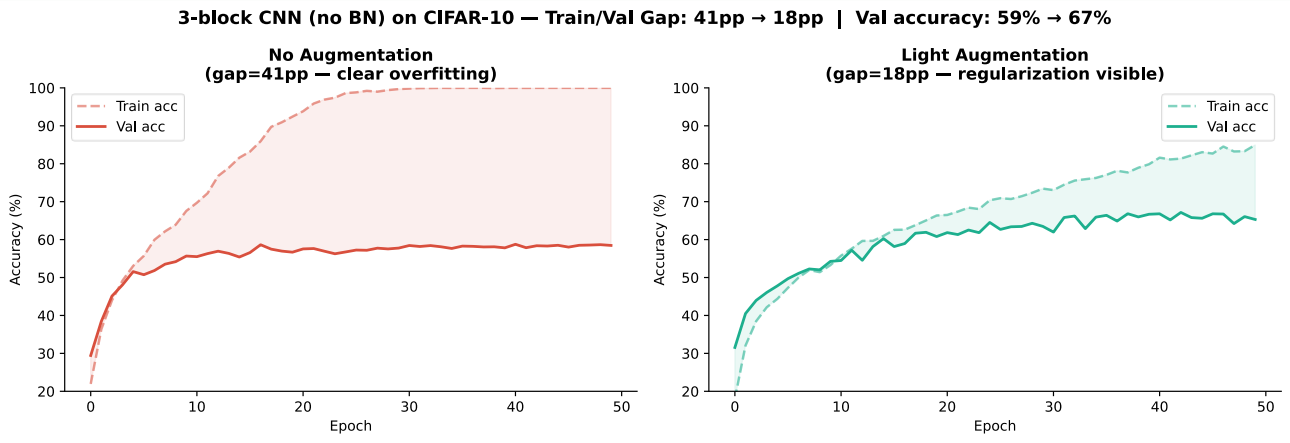
```python
# Step 2b: bake into the model
# (preferred — portable, no mismatch risk)
model = tf.keras.Sequential([
    tf.keras.layers.Input(
        shape=(32, 32, 3)),
    norm_layer,   # normalisation here
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(
        256,
        kernel_initializer='he_normal'
    ),
    tf.keras.layers.Activation('relu'),
    tf.keras.layers.Dense(10,
        activation='softmax'),
])
# Normalization runs at inference too
# No separate test-time normalization
```

03

Data Augmentation

Geometric · Colour · Keras layers · Training vs inference

Why Data Augmentation? — Implicit Regularization



How augmentation reduces overfitting

- **Expands effective dataset**
  Each epoch the model sees different versions → more training signal
- **Encourages invariance**
  Flipped cats are still cats → model generalises better
- **Low cost**
  No extra labels — transforms computed on-the-fly
- **Inference unchanged**
  At test time, original image is used
- **Acts as regularization**
  Input noise → similar effect to Dropout in the data space

Image Augmentation — Keras Transforms on a 256×256 Image

Image Augmentation Transforms — Same Keras Layers as Section 4

- Geometric (flip, rotation, zoom, translation): preserve class identity. Bottom-right 'Light augment' = exact config used in Section 4 notebook.
- Colour (brightness, contrast, hue): simulate varying lighting and camera conditions.

⚠ Always verify: some transforms break class invariance (e.g. flipping '6'/'9', rotating directional signs).

Keras Augmentation Layers — Practical Code


```python
# Augmentation pipeline
augment = tf.keras.Sequential([
    # Geometric transforms
    tf.keras.layers.RandomFlip(
        'horizontal'
    ),
    tf.keras.layers.RandomRotation(
        factor=0.10   # ±36 degrees
    ),
    tf.keras.layers.RandomZoom(
        height_factor=0.10
    ),
    tf.keras.layers.RandomTranslation(
        height_factor=0.1,
        width_factor=0.1
    ),
    # Colour transforms
    tf.keras.layers.RandomBrightness(
        factor=0.2
    ),
    tf.keras.layers.RandomContrast(
        factor=0.2
    ),
])
```

```python
# Apply ONLY during training
def preprocess(image, label,
               training=True):
    image = tf.cast(
        image, tf.float32
    ) / 255.0
    if training:
        image = augment(
            image, training=True
        )
    return image, label

train_ds = (
    ds.map(preprocess,
        num_parallel_calls=AUTOTUNE)
    .batch(32)
    .prefetch(AUTOTUNE)
)
val_ds = (
    ds_val.map(
        lambda x, y: preprocess(
            x, y, training=False),
        num_parallel_calls=AUTOTUNE)
    .batch(32)
    .prefetch(AUTOTUNE)
)
```

Augmentation: Training vs Inference

Training (training=True)

- Augmentation layers apply random transforms
- Each epoch: different version of each image
- Effectively multiplies dataset size
- Stochasticity acts as regularization
- Normalization always applied (deterministic)

```
model.fit(train_ds, ...)
# passes training=True automatically
```

Inference (training=False)

- Augmentation layers are disabled automatically
- Original image passed through unchanged
- Only deterministic preprocessing applied
- No stochasticity → reproducible predictions
- In standard Keras workflows, handled automatically

```
model.predict(val_ds, ...)
# passes training=False automatically
```

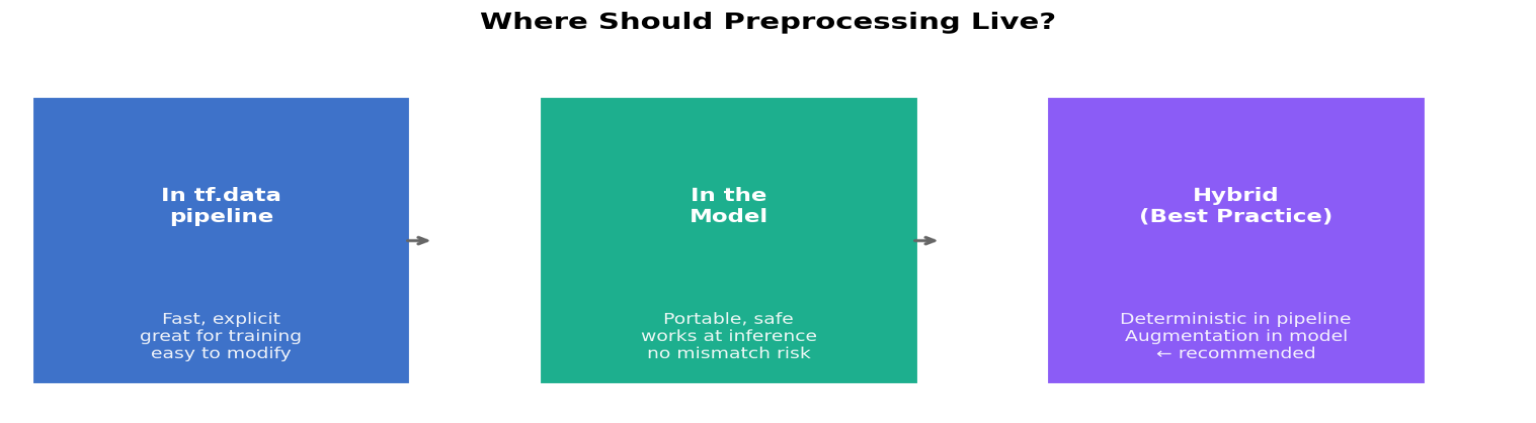
Key: augmentation layers check the `training` flag. model.fit() sets it True; model.predict() sets it False. In custom training loops, pass explicitly.

04

Keras Preprocessing

Where to put it · Oxford Flowers pipeline examples · TFRecord

Where Should Preprocessing Live?



In tf.data pipeline

✓ Fast — CPU-parallel; easy to inspect and modify independently

✗ Risk of train/inference mismatch if val_ds forgets a step

In the model

✓ Portable — preprocessing travels with the model; no separate test step

✗ Slightly slower if heavy transforms run on GPU; harder to iterate

Hybrid ← recommended

✓ Deterministic steps in pipeline; stochastic augmentation in pipeline or as first model layers — both valid

✗ Two places to maintain; augmentation needs training=True/False discipline

← Best practice for most tasks

Practical note: in this notebook, augmentation lives in the pipeline (map after cache). Both pipeline and model placements are valid.

End-to-End Preprocessing — Oxford Flowers

Raw JPEG bytes → Decode & Resize 224×224 → Normalize ÷255 → [0,1] → Augment (train only) → batch(32) + prefetch → MobileNet (Lecture 10)

```python
import tensorflow_datasets as tfds

IMG_SIZE, BATCH = 224, 32

def preprocess(image, label,
               training=True):
    image = tf.image.resize(
        image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(
        image, tf.float32) / 255.0
    if training:
        image = augment(
            image, training=True)
    return image, label
```

```python
(train_raw, val_raw), info = tfds.load(
    'oxford_flowers102',
    split=['train', 'validation'],
    as_supervised=True,
    with_info=True,
)
AUTOTUNE = tf.data.AUTOTUNE
n_train = info.splits['train'].num_examples

train_ds = (
    train_raw
    .map(preprocess,
         num_parallel_calls=AUTOTUNE)
    .cache().shuffle(n_train)
    .batch(BATCH).prefetch(AUTOTUNE)
)
val_ds = (
    val_raw
    .map(lambda x,y: preprocess(
             x, y, training=False),
         num_parallel_calls=AUTOTUNE)
    .batch(BATCH).prefetch(AUTOTUNE)
)
```

TFRecord — Efficient Storage for Large Datasets

For very large datasets (ImageNet, medical imaging), loading many small JPEG/PNG files is slow.
TFRecord packs samples into large binary files → faster sequential reads, less filesystem overhead.

Why TFRecord?

- Single large file → fewer filesystem seeks
- Supports GZIP/ZLIB compression
- Supports sharding for distributed training
- Compatible with AUTOTUNE prefetching
- Used by all major TF pretrained releases

```
feature_desc = {
    'image': tf.io.FixedLenFeature(
        [], tf.string),
    'label': tf.io.FixedLenFeature(
        [], tf.int64),
}

def parse_fn(serialized):
    f = tf.io.parse_single_example(
        serialized, feature_desc)
    img = tf.io.decode_jpeg(f['image'])
    return img, f['label']

ds = (
    tf.data.TFRecordDataset(filenames)
    .map(parse_fn,
         num_parallel_calls=AUTOTUNE)
    .batch(32)
    .prefetch(AUTOTUNE)
)
```

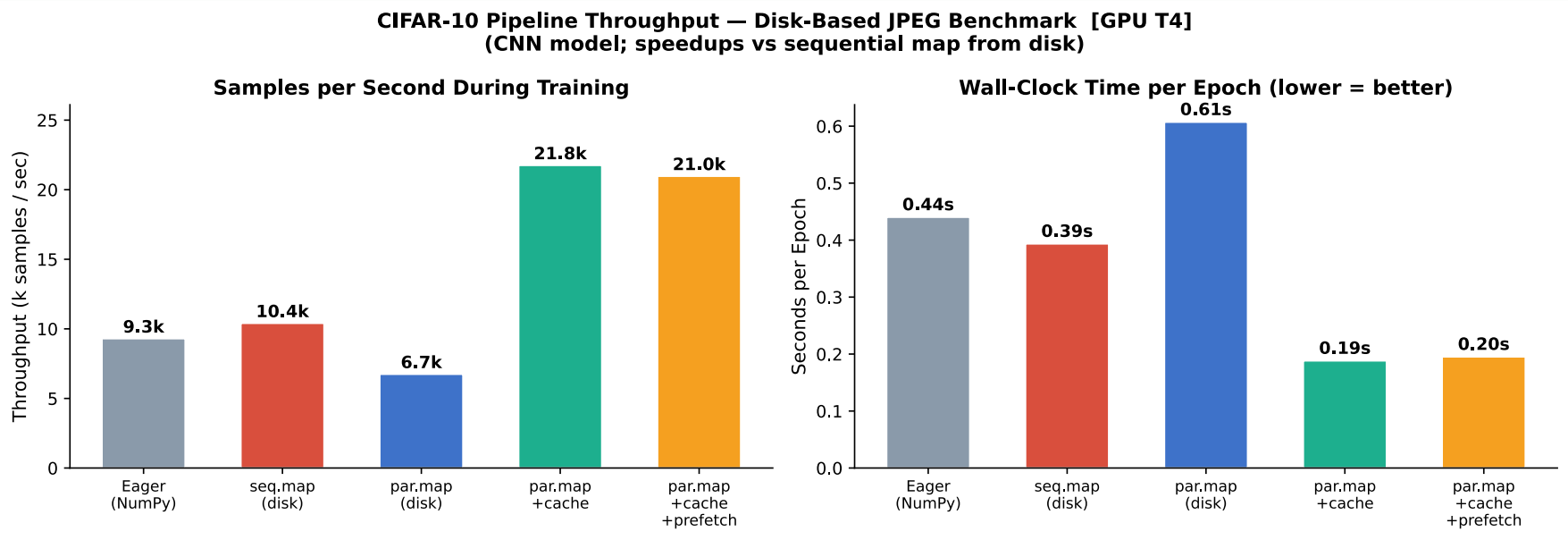
Practical rule: use TFRecord when dataset > 10 GB or disk I/O is the bottleneck. CIFAR-10/Flowers → from_tensor_slices is fine.

05

Experiments

Pipeline throughput · Normalization effect · Augmentation on CNN

Pipeline Throughput — Disk-based JPEG Benchmark

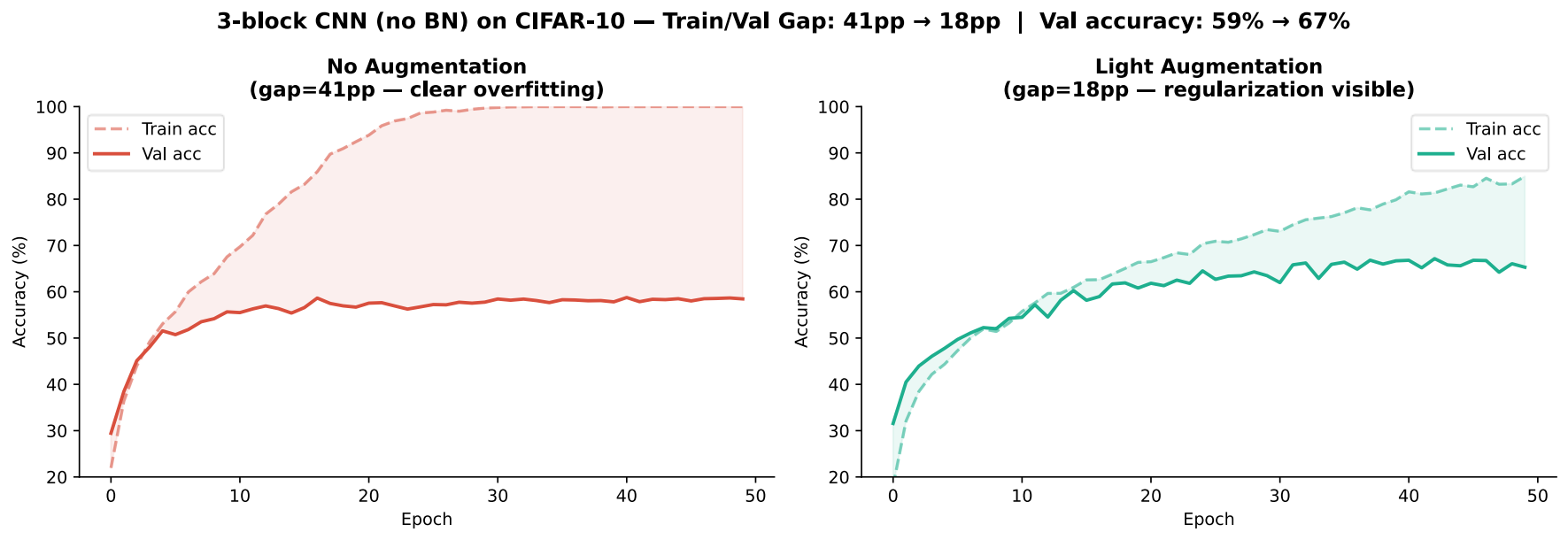


Key finding: cache() gives the largest single gain (2.2× over seq.map) by eliminating disk I/O from epoch 2 onward. par.map adds little for small 32×32 images. **prefetch is always best practice**; here cache() dominates the gain.

- Throughput gains reduce wall-clock training time; they do NOT improve val accuracy
- cache() when the deterministic part fits in memory — eliminates disk I/O from epoch 2
- prefetch(AUTOTUNE) is always low-cost best practice; larger benefit with bigger images or heavier models

Augmentation Experiment — CIFAR-10 Small CNN

Small CNN (3 conv blocks + Flatten head, no BN) on 6k CIFAR-10 training images. Preview of Lecture 7.



No augmentation
val=58.8% · train=100% · gap=41pp

Light augmentation (flip + translation + contrast)
val=67.2% · train=85% · gap=18pp

Augmentation as regularization
Gap: 41pp → 18pp · Val +8.4pp · Reduces overfitting, improves generalisation

Practical Guide — Data Pipeline Checklist

1. **Normalize inputs**
   Always. ÷255 for image baselines. z-score per channel for ImageNet-pretrained backbones.

2. **Add prefetch(AUTOTUNE)**
   Always the last step. Low cost, often helps; most impactful when GPU compute time >> CPU prep time.

3. **Shuffle before batch**
   shuffle(N).batch(B), not batch(B).shuffle(N). Shuffling batches does not randomise samples within them.

4. **Apply augmentation only at training time**
   Pass training=True/False. In custom loops, pass explicitly. Keras fit/predict handles it in standard workflows.

5. **Cache after deterministic preprocessing, before augmentation**
   .cache() after decode/resize/normalize when data fits in memory. Augmentation runs fresh on cached images each epoch.

6. **For transfer learning: use backbone-specific preprocessing**
   Each pretrained model (MobileNet, ResNet, EfficientNet) expects specific normalization — use its preprocess_input().

QUICK QUIZ

You are training a CNN on Oxford Flowers (8 189 images). GPU utilisation sits at 20–30% and training is slow. Loss is decreasing correctly and gradient norms look healthy. What is the most likely cause and best fix?

A The model is too small. Add more layers to increase GPU utilisation.

B The learning rate is too high. Lower LR to stabilise training and GPU load.

C The data pipeline is the bottleneck — the CPU cannot preprocess batches fast enough.
Fix: add prefetch(AUTOTUNE), cache(), and num_parallel_calls=AUTOTUNE to map().

D BatchNorm is causing instability. Remove it to improve GPU efficiency.

Hint: GPU% < 50% with healthy loss + gradients almost always means data pipeline bottleneck.

Answer: C — Data pipeline bottleneck

Root cause:

GPU sits idle (20–30%) waiting for the CPU to deliver the next batch. Loss and gradients are fine — the problem is data throughput, not optimization.

Why C works:

prefetch(AUTOTUNE) overlaps CPU preprocessing with GPU training. cache() eliminates repeated disk I/O. num_parallel_calls=AUTOTUNE spreads map() across CPU cores.

Why not A:

Adding layers increases compute per batch — but the real bottleneck is data delivery, not model size.

Why not B:

LR affects convergence quality, not pipeline throughput. Lowering LR makes training take more epochs, not fewer.

Why not D:

BatchNorm has no meaningful effect on CPU/GPU I/O. Removing it would hurt training stability without fixing the bottleneck.

Pattern: GPU% < 50% + healthy loss → data pipeline bottleneck → prefetch / cache / parallel map

Key Experimental Results — CIFAR-10

Section 3 — Normalization (SGD lr=0.05, MLP without BN, 12k subset, 20 epochs)		
Raw pixels (0–255)	10.4%	Fails — large gradient magnitudes overwhelm SGD
÷ 255 → [0, 1]	42.7%	Stable convergence; not centered (mean ≠ 0)
z-score per channel ✓	43.9%	Fastest convergence — centered, unit variance; ImageNet standard

Section 4 — Augmentation (CNN, Flatten head, no BN, Adam lr=1e-3, 6k subset, 50 epochs)		
No augmentation	58.8%	Train=100% · Gap=41pp ← strong overfitting
Light augmentation ✓	67.2%	Train=85% · Gap=18pp ← gap −23pp · val +8.4pp

Section 5 — Throughput (disk JPEG, small CNN benchmark, Colab T4 GPU)		
seq.map from disk (baseline)	9.8k	samp/sec · 0.42s/epoch
par.map + cache ✓	21.4k	samp/sec · 0.19s/epoch · 2.2× seq.map ← cache is the key gain
par.map + cache + prefetch	15.1k	samp/sec · 0.27s/epoch · prefetch may reduce throughput for small images/light models

Lecture 6 Summary

Key Takeaways

tf.data Pipeline

map → shuffle → batch → prefetch. AUTOTUNE everywhere. cache() after deterministic steps, before augmentation.

Normalization

Always normalize. ÷255 for baselines; z-score per channel (ImageNet style) for pretrained backbones. Bake into model.

Augmentation

Reduces overfitting gap (41pp→18pp on 6k CIFAR-10) and improves val accuracy (+8.4pp). Training only — free regularization.

Preprocessing Location

Hybrid is best: deterministic transforms in tf.data, stochastic augmentation in pipeline or as first model layers.

Practical Default Stack

cache()→shuffle()→map(augment)→batch(32)→prefetch(AUTOTUNE)+÷255. cache() gave 2.2× speedup; always add prefetch(AUTOTUNE) as best practice.

Next: Lecture 7

CNN Fundamentals — convolutional filters, pooling, spatial invariance, and CIFAR-10 with CNNs.

Next: Lecture 7 — CNN Fundamentals · Filters · Pooling · CIFAR-10 with CNNs · Geron Ch. 14

Mixed Precision Training — float16 on GPU

Modern GPUs have Tensor Cores for float16. Mixed precision runs forward/backward in float16 but keeps master weights in float32 → 2–3× speedup with no accuracy loss on compatible hardware.

float32 (default)

- Full precision
- No special setup needed
- Use when debugging numerics

```
# Default — no action needed
```

float16 (activations)

- 2× faster on Tensor Cores
- Risk: gradient underflow
- Mitigated by loss scaling

```
policy = 'mixed_float16'
mixed_precision.set_global_policy(policy)
```

bfloat16 (TPU/A100)

- Same range as float32
- No loss scaling needed
- Best for TPU, A100+

```
policy = 'mixed_bfloat16'
mixed_precision.set_global_policy(policy)
```

```
from tensorflow.keras import mixed_precision
```